

japp — Complete Setup Guide

Set up your own automated job-search assistant, from a blank laptop to daily job emails. No coding experience needed. Budget about an hour, mostly for creating accounts.

What you're setting up

japp runs quietly on your laptop and does four things:

1. **Finds jobs for you.** Every hour it checks job boards for newly posted roles that match what you're looking for.
2. **Filters hard.** An AI reads every posting and only keeps jobs you could realistically get an interview for. No spam — on a day with nothing good, you get no email at all.
3. **Emails you the good ones.** Each email shows the job, why it matched, what's honestly missing, and two buttons: one to view the posting, one that says "**Tailor my resume for this job.**"
4. **Tailors your resume on request.** Tap that button, hit send on the reply it opens, and within the hour a version of *your* resume tailored to that exact job lands back in your inbox as a Word file. Everything in it comes from your real experience — the AI is not allowed to make things up. You read it, then apply.

What it costs

- **Anthropic API credits** (the AI): \$5–10 lasts weeks. Scoring a job costs fractions of a cent; tailoring a resume about 25 cents.
- Everything else — the code, the tools, Gmail — is free.

Your safety net for every step below: once you finish Part 2, you'll have **Claude Code** installed — an AI assistant that runs on your laptop and can see this project. If you get stuck *anywhere*, open the japp folder in Claude Code and type what's wrong in plain English ("I got this error when I ran uv sync", "help me fill in profile.yaml"). It will walk you through it.

Part 1 — Install the tools (one time, ~10 minutes)

Step 1.1 — Open a terminal.

Press the **Windows key**, type `powershell`, press Enter. A blue or black window opens where you type commands. (Mac: open the **Terminal** app.) Keep it open for all of Part 1.

Step 1.2 — Install Python.

```
winget install Python.Python.3.12
```

Press Enter and wait for it to finish. (Mac: install from python.org/downloads.)

Step 1.3 — Install uv (the tool that runs the project).

```
winget install astral-sh.uv
```

(Mac: `brew install uv`.)

Step 1.4 — Install Claude Code (your AI helper).

Go to claude.com/claude-code and follow the install instructions for your computer. You'll sign in with a Claude account — create one if you don't have it (the free account works; you'll add API credits separately in Part 3).

Step 1.5 — Close the terminal and open a new one.

This makes the newly installed tools available. Quick check — type:

```
uv --version
```

If a version number appears, Part 1 is done.

Part 2 — Get the code (~5 minutes)

Step 2.1 — Download it.

1. Go to github.com/jeremiahtswett/japp
2. Click the green <> **Code** button → **Download ZIP**
3. Open your Downloads folder, right-click `japp-main.zip` → **Extract All...**
4. Move the extracted `japp-main` folder somewhere permanent, e.g. your **Documents** folder. You can rename it to just `japp`.

Step 2.2 — Point your terminal at the folder.

In a new terminal, type `cd` (with a space), then drag the japp folder from File Explorer onto the terminal window, and press Enter. Or type the path directly:

```
cd "$HOME\Documents\japp"
```

Every command in the rest of this guide is typed in a terminal that's "in" this folder. If you close the terminal, just repeat this step.

Step 2.3 — Install the project's dependencies.

```
uv sync
```

Step 2.4 — Create your personal config files.

```
uv run japp init
```

This creates three files you'll fill in during Parts 3 and 4: `profile.yaml`, `sources.yaml`, and `.env`. They stay on your laptop only — they're never uploaded anywhere.

Part 3 — Create accounts and keys (~20 minutes)

3A. Anthropic API key (the AI that scores jobs and tailors your resume)

1. Go to console.anthropic.com and sign in (same Claude account from Step 1.4 works).
2. Click **Billing** → add a payment method → buy **\$10** of credits. (You can turn off auto-reload.)
3. Click **API Keys** → **Create Key** → name it `japp` → **copy the key immediately** (it starts with `sk-ant-` and is only shown once).

3B. Gmail App Password (lets japp send and read email for you)

japp emails *you* from *your own* Gmail account, and reads the "tailor my resume" replies you send back. It needs a special app password — **not** your normal password:

1. Go to myaccount.google.com/security
2. Turn on **2-Step Verification** if it isn't already (required).
3. Go to myaccount.google.com/apppasswords (or search "app passwords" in the account settings search bar).
4. App name: `japp` → **Create** → copy the 16-character password (ignore the spaces).

3C. Adzuna (free, optional but recommended — broad job-board search)

1. Register at developer.adzuna.com (free).
2. Copy your **Application ID** and **Application Key** from the dashboard.

3D. Put all the keys in the .env file

In File Explorer, open the japp folder and double-click `.env` (choose Notepad if asked; if you don't see it, enable View → Show → File name extensions and Hidden items). Fill in:

```
ANTHROPIC_API_KEY=sk-ant-...your key from 3A...

SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=youraddress@gmail.com
SMTP_PASSWORD=...the 16-character app password from 3B, no spaces...
DIGEST_TO_EMAIL=youraddress@gmail.com

ADZUNA_APP_ID=...from 3C...
ADZUNA_APP_KEY=...from 3C...
```

SMTP_USER and DIGEST_TO_EMAIL are both your own Gmail address. japp sends the job emails from your account to your account, and only accepts "tailor" requests coming from that same address — nobody else can trigger anything.

Part 4 — Tell it about you (~15 minutes)

Step 4.1 — Fill in profile.yaml.

Open `profile.yaml` from the japp folder in Notepad. Every field has instructions and a worked example right above it. Replace every `REPLACE ME`. The important ones:

- `target_titles` — every job title you'd apply to, e.g. "Data Analyst", "Marketing Coordinator", "HR Coordinator".
- `locations` — e.g. "New York, NY" and "Boston, MA".
- `years_of_experience` — years of experience *in the field you're applying to*. Changing careers? Put `0` — the AI then hunts for genuinely entry-level roles you can actually land.
- `education` — e.g. "B.A., Boston College".
- `skills_summary` — a few honest sentences about what you've actually done. Managing a store counts: hiring, scheduling, inventory, marketing, customer service — write it plainly. **Don't exaggerate**; honest matching is the whole point.

Step 4.2 — Fill in sources.yaml.

Open `sources.yaml`. Easiest start: set `adzuna → enabled: true` and set its `what` / `where` to your role and city. You can also list specific companies' job boards — the comments in the file show how.

Step 4.3 — Drop in your resume.

Copy your resume (one `.docx` or `.pdf` file) into the `corpus` folder inside `japp`. Then run:

```
uv run japp parse-resume
```

This reads your resume into `corpus\experience_corpus.yaml` — the complete list of facts the resume-tailor is allowed to use. Open that file and check it: fix anything it misread, and add extra truthful bullet points that didn't fit on your resume (more material = better tailoring).

Step 4.4 — Check everything.

```
uv run japp status
```

It tells you exactly what's still missing. When it prints **Config OK**, do the first real run:

```
uv run japp run
```

First run takes a few minutes. If good matches exist, the digest email arrives.

Part 5 — Make it run automatically (two commands)

Two commands, typed in the terminal while it's in the `japp` folder. They create two Windows scheduled tasks that run in the background — **no terminal window needs to stay open**, you just need to be logged in.

Task 1 — the full job search, every hour:

```
schtasks /Create /TN "japp" /SC HOURLY /MO 1 /TR "cmd /c cd /d %CD% && uv run j  
app run"
```

Task 2 — check for your "tailor my resume" replies, every 10 minutes:

```
schtasks /Create /TN "japp-inbox" /SC MINUTE /MO 10 /TR "cmd /c cd /d %CD% && u  
v run japp inbox"
```

This second task is what makes the tailor button feel fast: when you tap it, your tailored resume comes back within about 10 minutes instead of waiting for the next hourly run. Checking the mailbox is free — the AI only does work (and costs anything) when a request is actually waiting.

To confirm both tasks exist: `schtasks /Query /TN "japp"` and `schtasks /Query /TN "japp-inbox"`.

A sleeping laptop pauses the schedule. If your emails seem to stop, your laptop was probably asleep or off — it catches up on the next run. Plug it in and set Settings → Power → "make my device sleep" to Never (when plugged in) for best results.

Your daily routine (the whole point)

1. A **[japp]** email arrives when there's a job worth your time. No email = nothing good today = the system is working, not broken.
2. Read the match reasons and the honest gaps. Interested? Tap **"Tailor my resume for this job."** Your mail app opens a pre-written reply — just hit **Send**.
3. Within about 10 minutes, a reply arrives with `tailored_resume.docx` attached and a summary of which job requirements it covers (and which it honestly doesn't).
4. **Read the resume before using it.** It's built only from your real experience, but you are the final check.
5. Tap the apply link in the original email and submit with your tailored resume. Done.

If something breaks

- Open the japp folder in **Claude Code** and describe the problem in plain English. It can read the project, the logs (`data\logs\japp.log`), and this guide.
- `uv run japp status` diagnoses most config problems and tells you what to fix.
- Every command can be tested safely with `--dry-run` added at the end — it shows what would happen without sending anything or spending anything.